



Sandboxing my AI Agents, One Layer at a Time

Juan Antonio "Ozz" Osorio
typing here and there @  **STACKLOK**

Juan Antonio “Ozz” Osorio

Mexican living in Finland

Background: security for OpenStack, Kubernetes, bare metal

"AI tooling should be secure by default, not secure by suffering"



How it started...

What is this AI thing?



Look at this MCP thing...

Enter ToolHive



I was the bottleneck

One ~~careful~~ paranoid human

Read the plan. Check the command. Decide whether to continue.

Do you want to proceed?

- 1. Yes
- 2. Yes, allow reading from `MyAwesomeProject/` during this session
- 3. No

Esc to cancel · Tab to add additional instructions

But models got better...

I'm just approving all the time now...



So... I moved the whole agent

Enter Brood Box

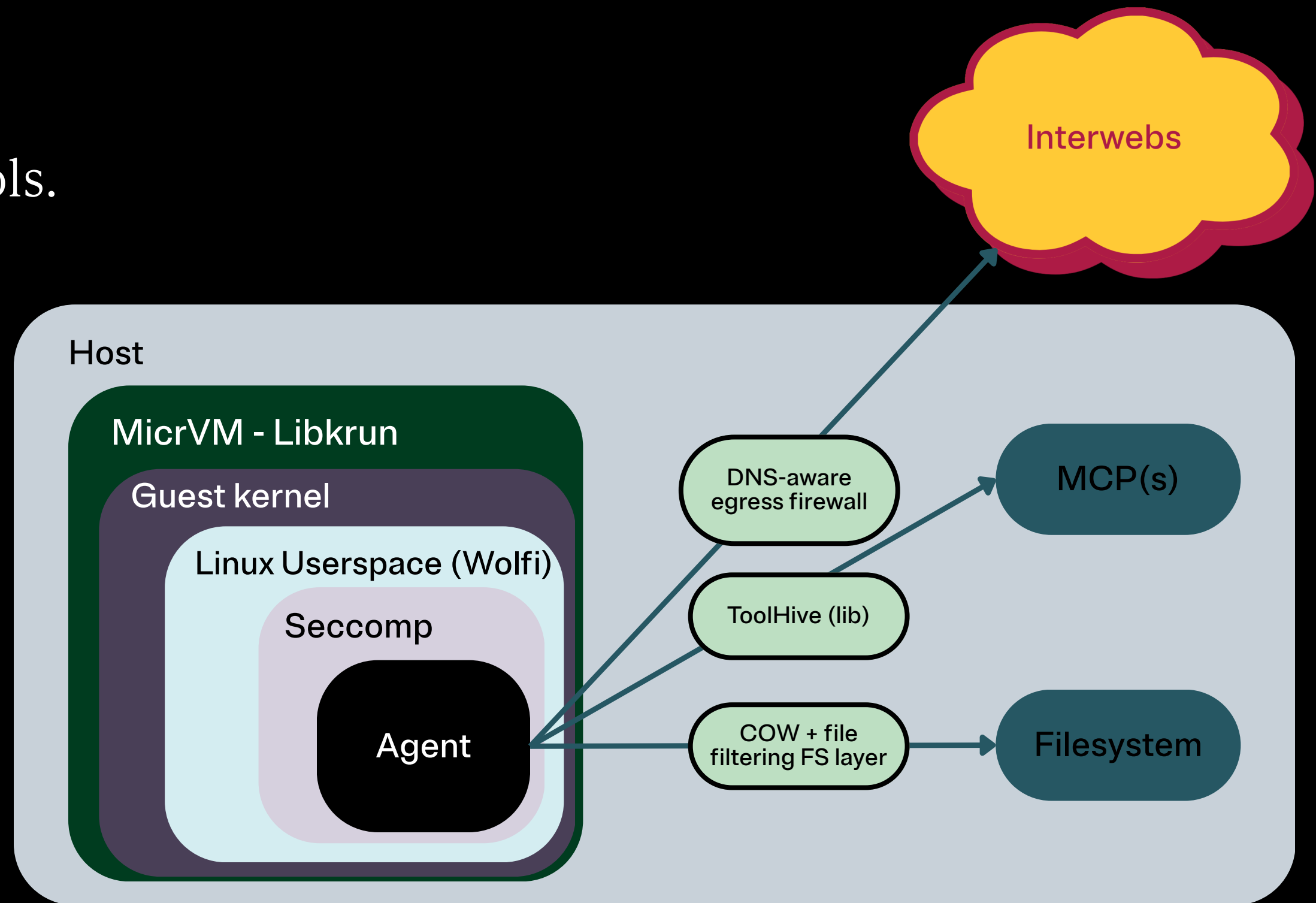
Contain the whole process

Hardware-isolated microVM
Copy-on-write workspace
Constrained connectivity (egress and credentials)

--dangerously-
skip-permissions

The box kept getting thicker

More autonomy. More controls.



What about scale?

What about the cloud?

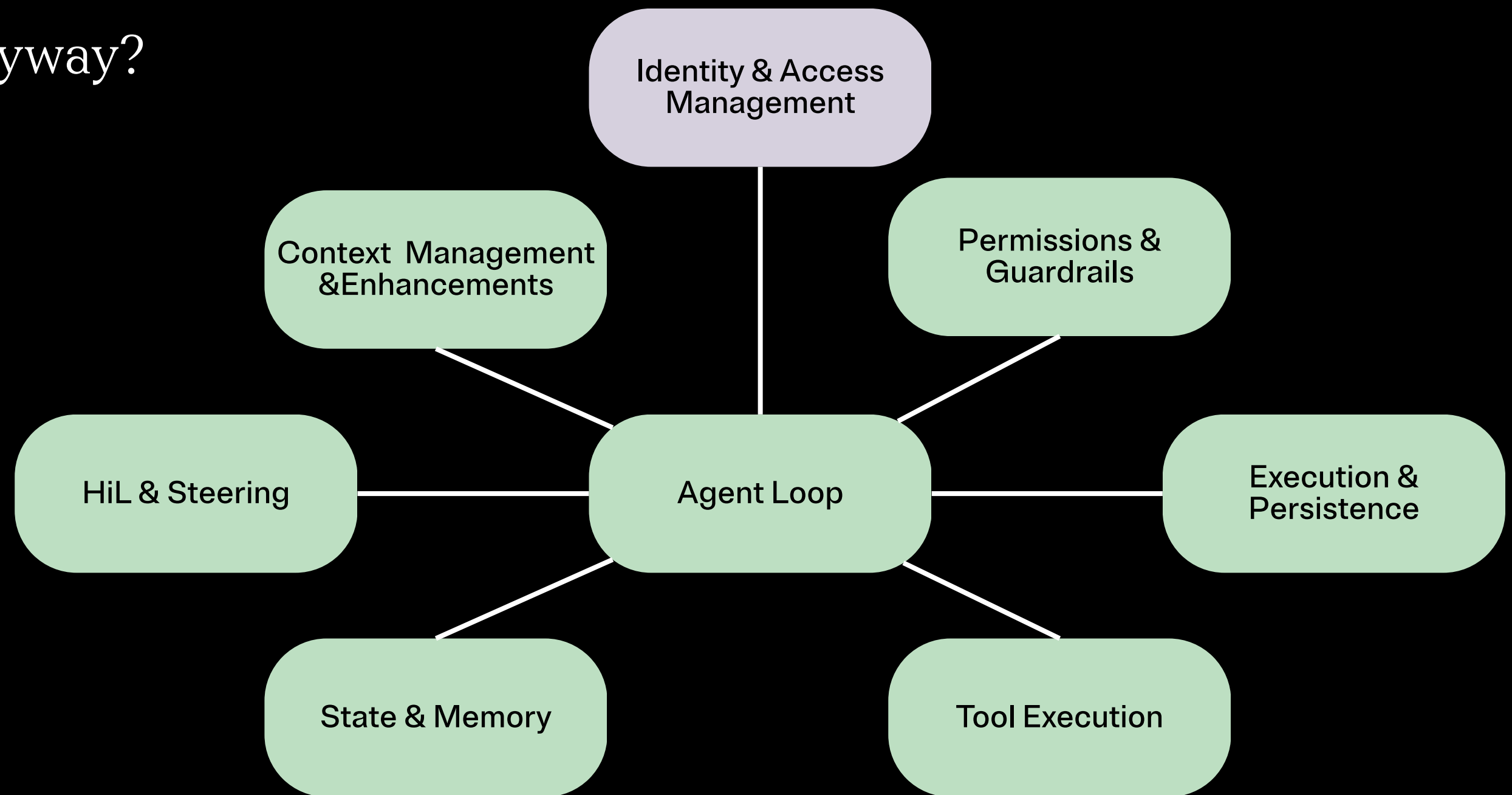
Are we just lifting and shifting?...



Is the agent really a black box?

What is a coding agent anyway?

Let's open it up!



The loop becomes infrastructure

The loop needs explicit contracts

File access + command execution

What is the agent able to see and run?

Identity + authorization

Who is acting, and what are they allowed to do?

Approvals + resume

How does work pause and safely continue?

Persistence + events

What survives restart and can be replayed?

Exploding the harness

I want to open the box... But which box?



Mecat1

An Open & Cloud Native Agent Harness



Workers are disposable

Durable sessions, pending approvals, replayable event history.

Execution is explicit

Workspace and shell only when needed. Tools and services without either.

Delegation cannot widen authority

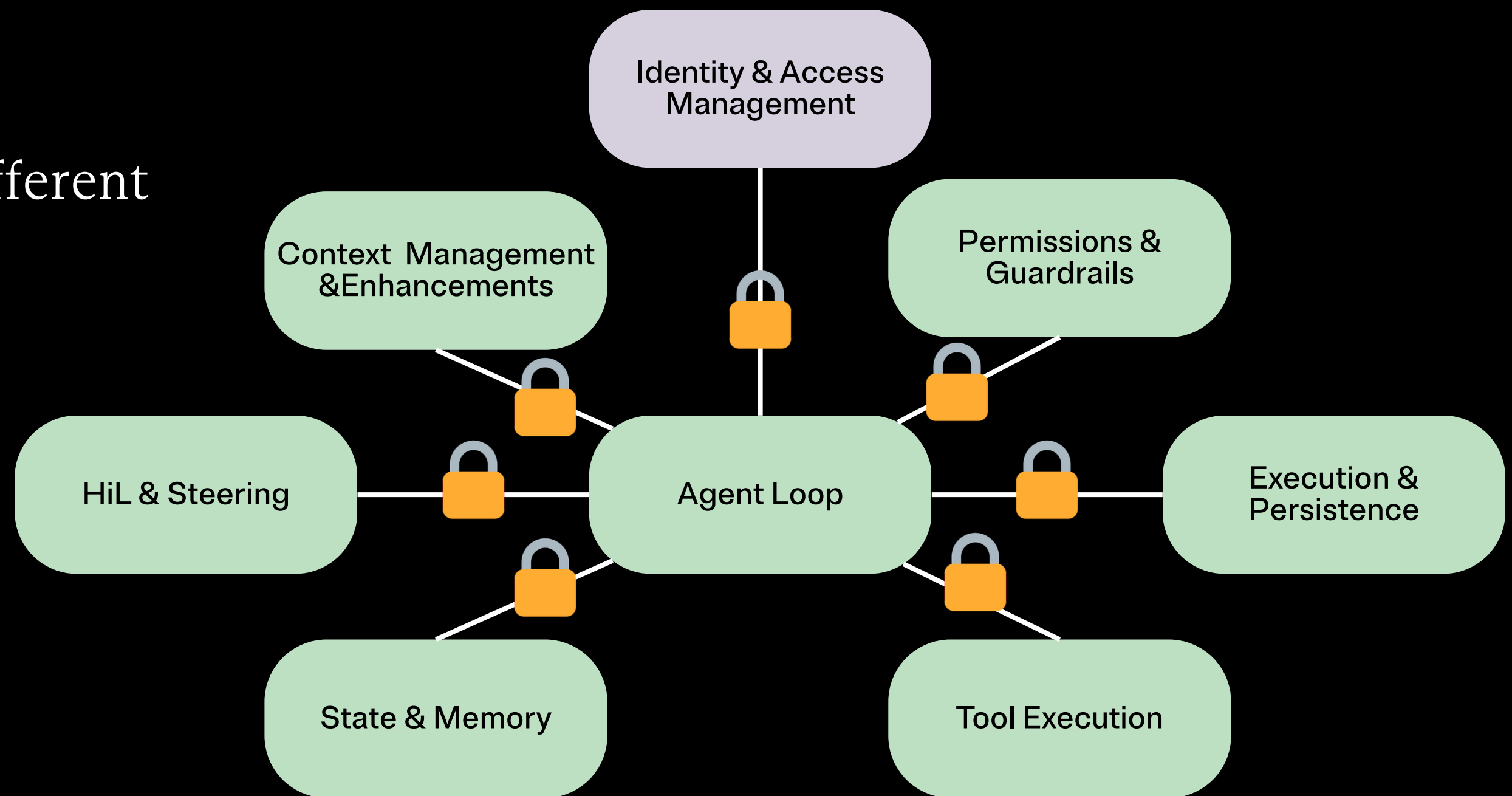
Subagents and teams stay within the capabilities their parent can grant.

The same engine, your infra

Embed in Golang or serve or APIs. Bring your own providers, stores and clients.

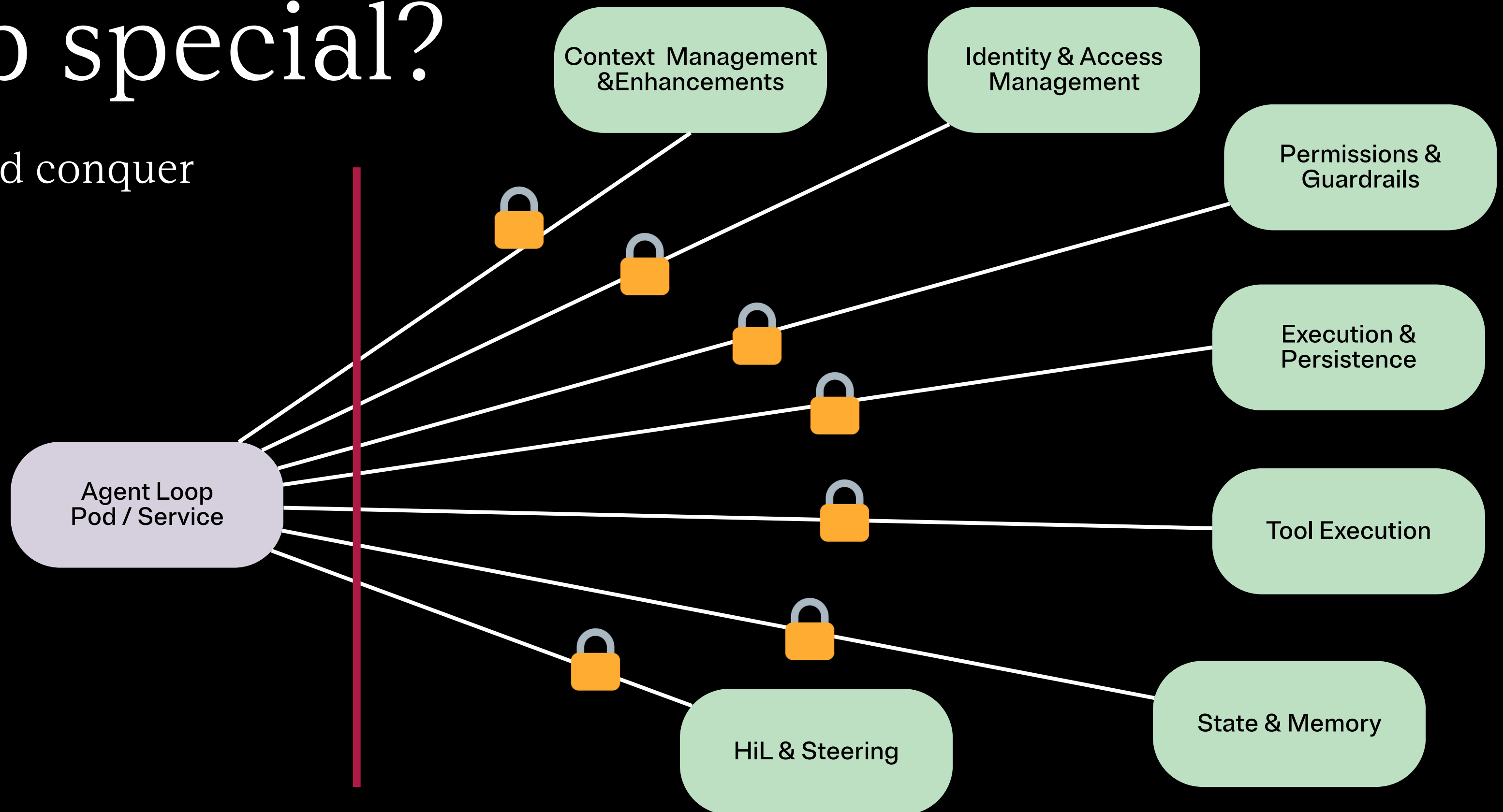
But how is this more secure?

Each component has a different boundary



Why is the loop special?

Divide and conquer



... But... What
about the
MicroVMs?



Separation of concerns leads to security

A smaller profile is a more safer one



Resources



STACKLOK



**ToolHive
@
GitHub**



**Brood Box
@
GitHub**



**Mecat1
@
GitHub**